Research Problem: 单卡 MoE 推理的存储需求

- **MoE 每次仅激活少数专家，但不同输入可能激活不同专家。单个推理任务可用的显存和 CPU 内存有限，无法容纳全部专家；当所选专家位于 HDD 时，必须读取并传输权重后才能计算。**

单卡仍需支持模型中的全部专家

本次路由	E1	E2	E3	E4	E5	E6	E7	E8

蓝色 E8 为本次激活的专家；其存储位置决定读取开销。

GPU 显存	E1	E2		直接计算
CPU 内存	E3	E4	E5	传输至显存
HDD	E6	E7	E8	从 HDD 读取并传输

E8：从 HDD 读取 → 经 CPU 内存传输 → 在 GPU 上计算

应用场景与研究任务

共享 GPU 服务器
任务仅分配一张 GPU 和部分内存

离线生成
问答、代码生成允许更长响应时间

单卡工作站
权重规模超过任务可用内存

MECC：MoE 专家缓存编译器
MoE Expert Cache Compiler
初始放置 · 联合缓存 · 动态调整

研究目标：在单卡上运行完整 MoE 模型，降低专家权重读取造成的延迟。

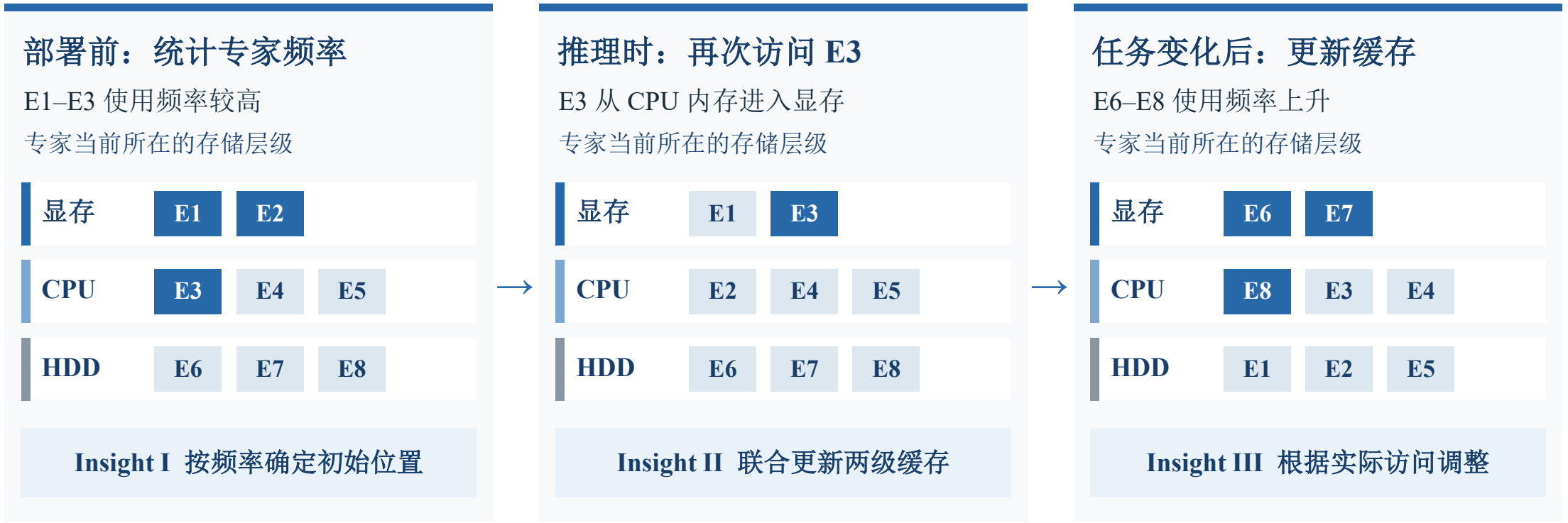
Related Work: 现有方法与 MECC 的对比

- 现有方法支持跨显存、CPU 内存和 HDD 读取专家，并缓存常用专家。在共享服务器中，还需要根据任务的内存预算确定初始放置、提前加载常用专家，并适应任务变化。

方法	显存/内存/HDD 三级存储	完整专家计算	按内存预算安排专家	提前加载常用专家	预测 HDD 读取需求	任务变化时更新缓存
SwapMoE[1]	✗	✗	✗	✓	✗	✓
Fiddler[2]	✗	✓	✗	✗	✗	✗
MoE-Infinity[3]	✗	✓	✗	✗	✗	✓
MoE-Lightning[4]	✗	✓	✗	✗	✗	✗
SSD-LLaMA[5]	✓	✓	✗	✗	✗	✓
MECC	✓	✓	✓	✓	✓	✓

Our Approach: MECC 的三项设计

- 部署前统计专家的使用频率，据此确定显存和 CPU 内存中的初始专家。推理过程中根据实际路由更新缓存。下图依次展示三项设计如何减少 HDD 读取。



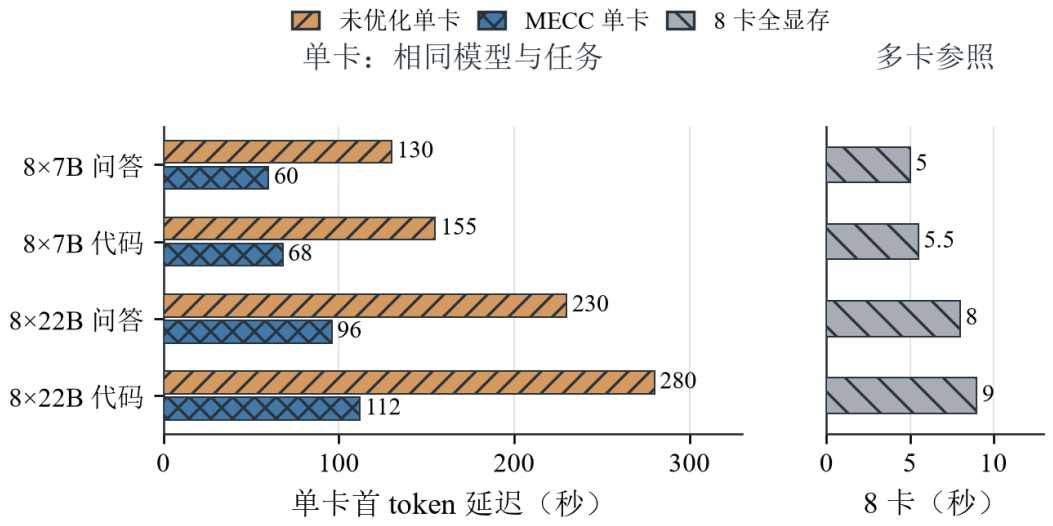
深蓝表示当前高频访问或被选中的专家；其他方块表示专家所在的存储层级。

HDD 保存完整专家权重；显存与 CPU 内存缓存当前更可能访问的专家。

Motivation I: 首次访问带来的读取延迟

- 当多卡总显存足以容纳模型时，专家可分散驻留在各卡显存，路由后传输 token 并执行。单卡的显存和任务内存预算有限，首次请求可能激活 HDD 上的专家。左图比较四组任务的首 token 延迟。

(a) 四组任务的首 token 延迟



首次访问 HDD 上的专家增加首 token 延迟。

(b) 两种部署的专家读取路径

多卡全显存：专家分散驻留在 GPU



选中 E8 → 跨卡传 token → 所在 GPU 执行

单卡：E8 仍在 HDD



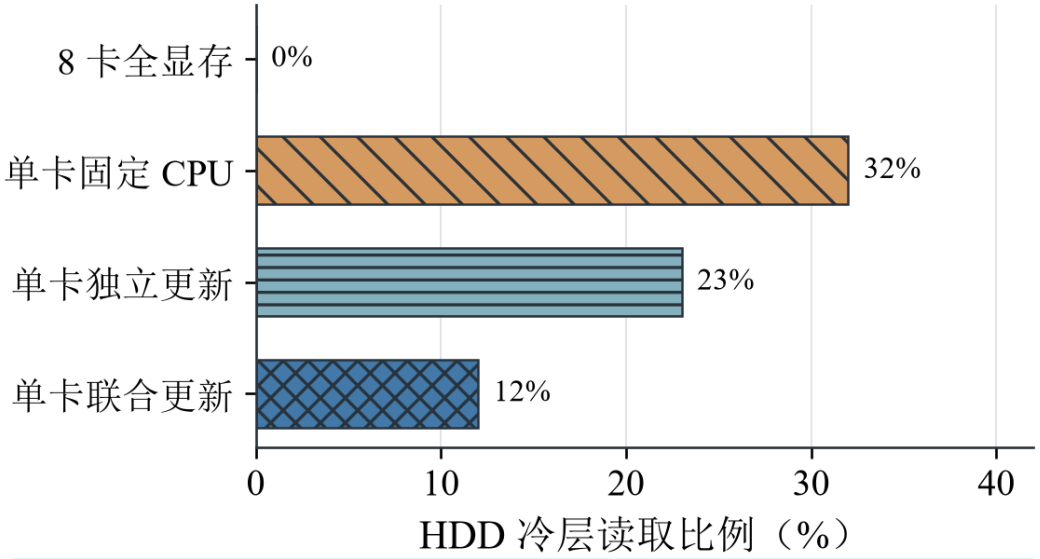
HDD 读取与权重传输均发生在专家计算之前。

问题①：推理开始前，哪些专家应优先进入显存和 CPU 内存？

Motivation II: 重复访问导致多次 HDD 读取

- 在多卡全显存配置中，再次访问同一专家无需读取 HDD。单卡的显存与 CPU 内存容量有限：E6 从 HDD 载入后可能被替换，再次访问仍需读取。左图比较不同缓存更新方式的读取比例。

(a) HDD 冷层读取比例



专家被缓存替换后，重复访问仍需读取 HDD。

(b) 重复访问 E6 的读取路径

多卡全显存：专家分散驻留



E6 → 其他专家 → E6：两次均命中显存

单卡：显存与 CPU 内存容量有限



读取 E6 → E6 被替换 → 再次读取 E6

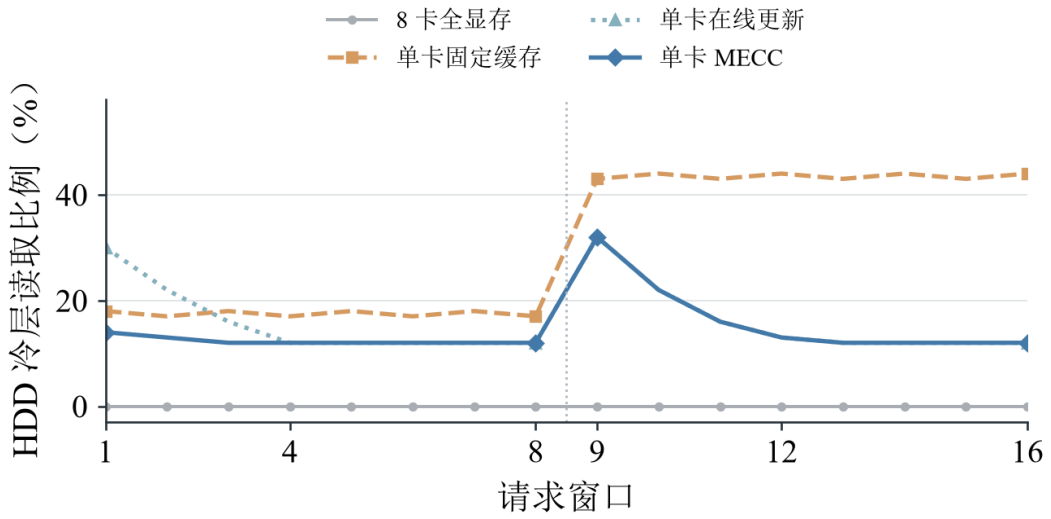
固定的 CPU 缓存无法保留被替换的 E6。

问题②：如何利用显存与 CPU 内存减少重复读取？

Motivation III: 任务变化引起专家访问变化

- 多卡全显存配置在任务变化后仍可直接访问各卡专家。单卡依据先前任务安排缓存；若新任务频繁访问另一组专家，而缓存未及时更新，HDD 读取比例会持续升高。左图显示切换前后变化。

(a) 任务切换前后的 HDD 读取比例



任务切换后，固定缓存仍保留旧任务的常用专家。

(b) 新任务常用专家的存储位置

多卡全显存：新任务专家仍在显存



任务 A → 任务 B：路由到 E6–E8 所在 GPU

单卡：缓存仍保留旧任务的常用专家



任务 B 频繁访问 E6–E8，却仍需从 HDD 读取。

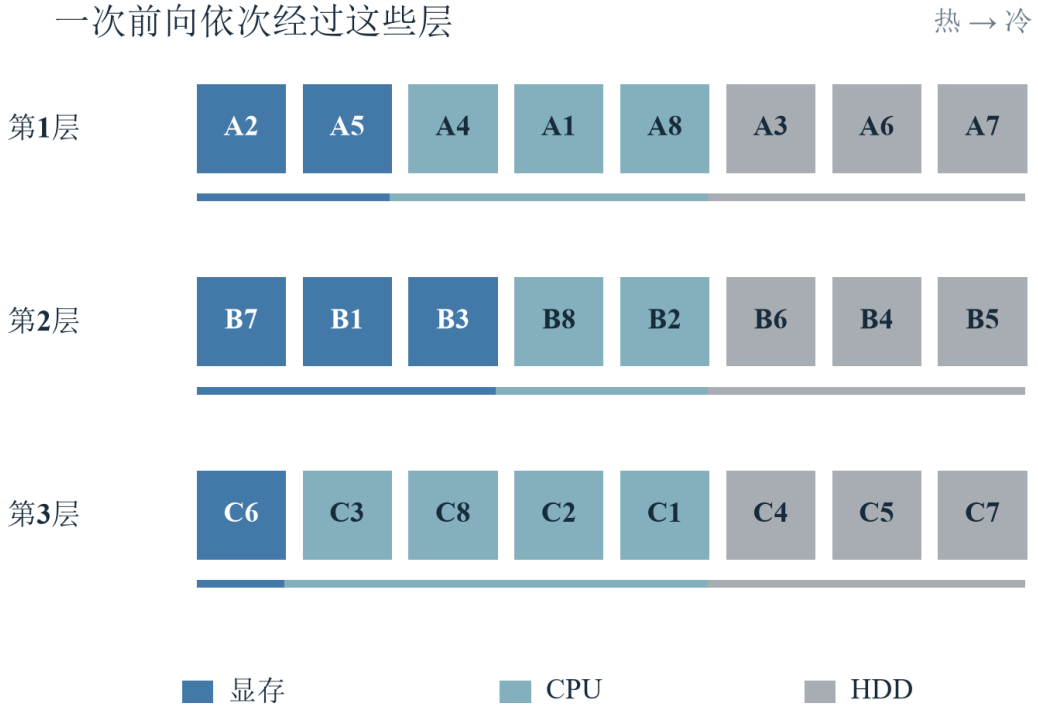
常用专家已改变，但缓存内容没有更新。

问题③：任务变化后，如何更新显存与 CPU 内存中的专家？

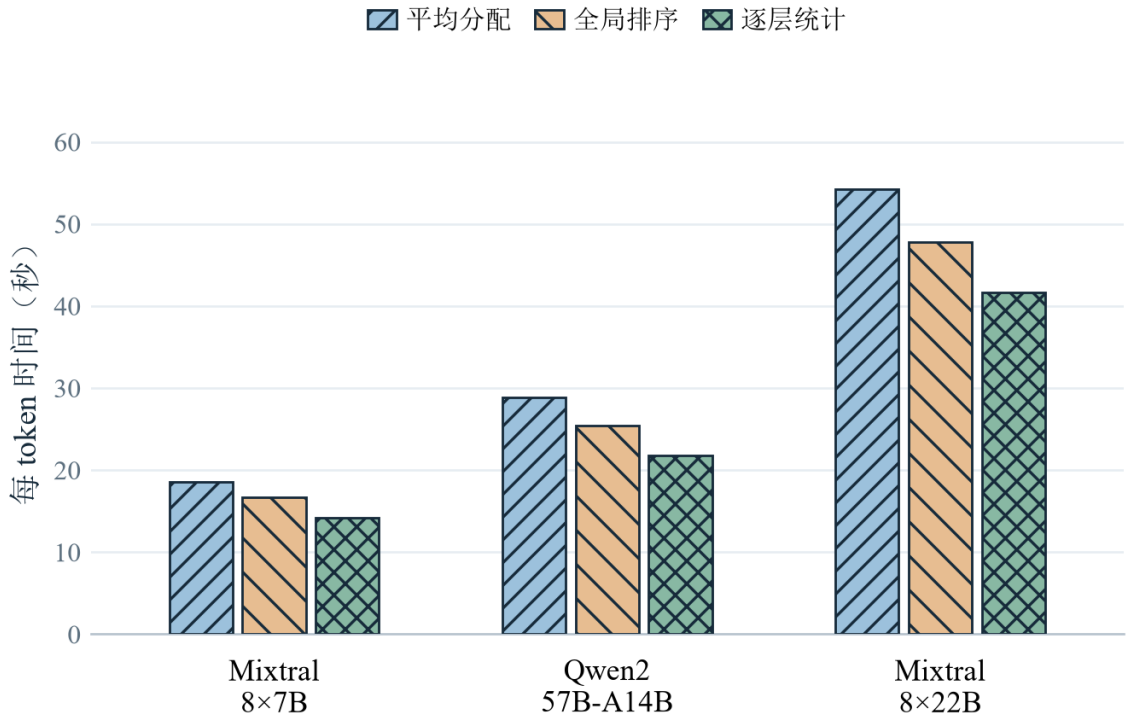
Insight I: 按专家访问频率确定初始位置

- 用代表性输入运行模型，统计每一层各专家被激活的次数。部署时分别对每层专家排序，在显存和 CPU 内存预算内优先缓存高频专家，其余权重保存在 HDD。

设计示意：分别统计每层专家



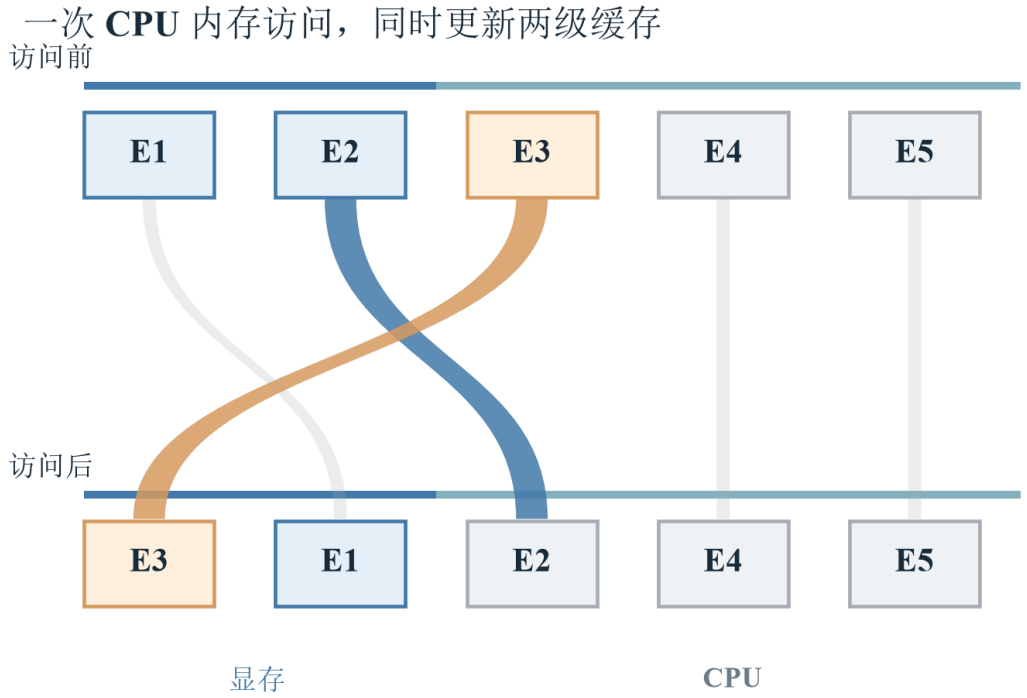
消融实验：比较不同初始放置方法



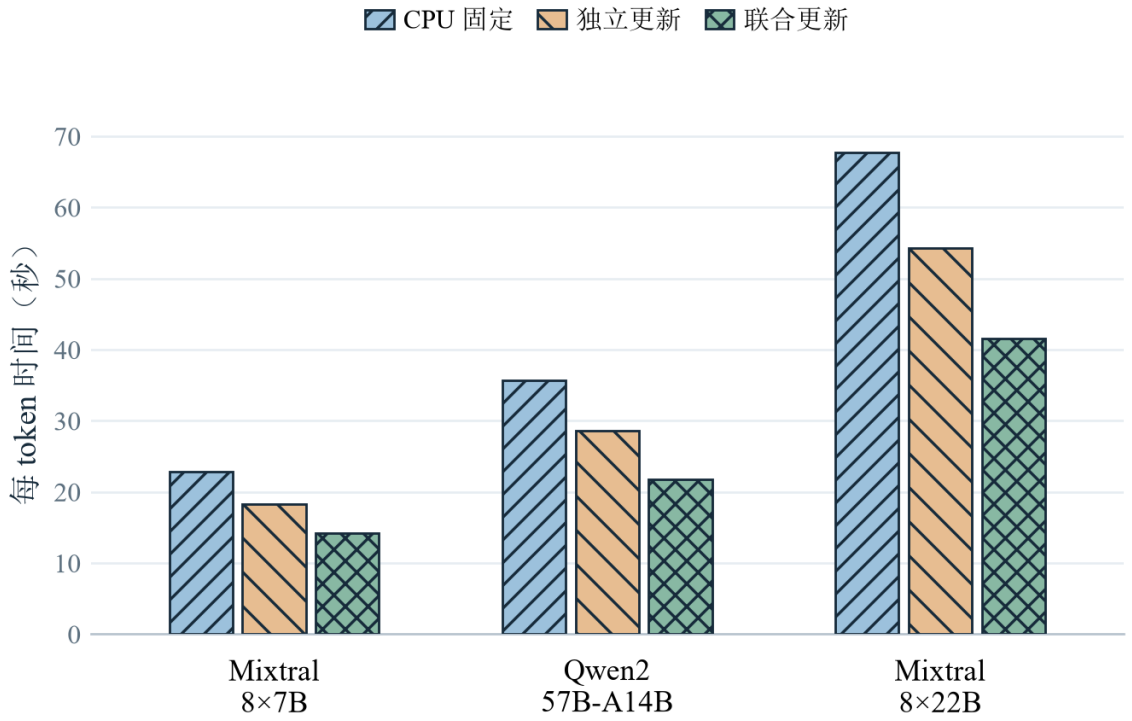
Insight II: 显存与 CPU 内存联合更新缓存

- 从 CPU 内存访问专家时，将其传输至显存；被替换的显存专家则保留在 CPU 内存中。
  这样可以同时更新两级缓存，降低再次访问该专家时读取 HDD 的概率。

设计示意：一次访问更新两级缓存



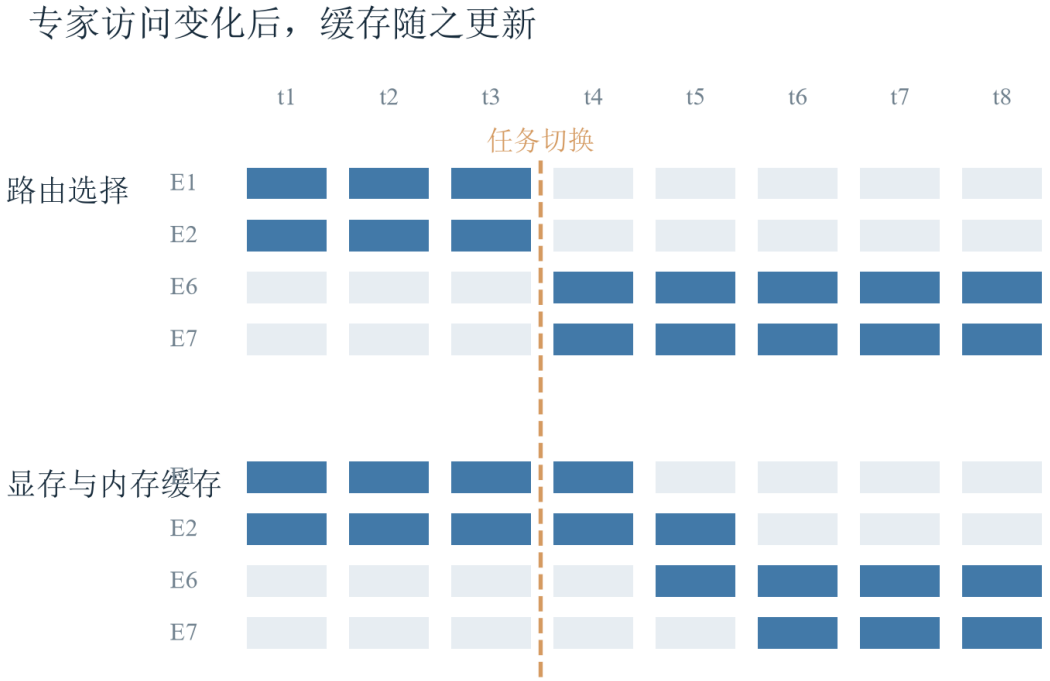
消融实验：比较不同缓存更新方法



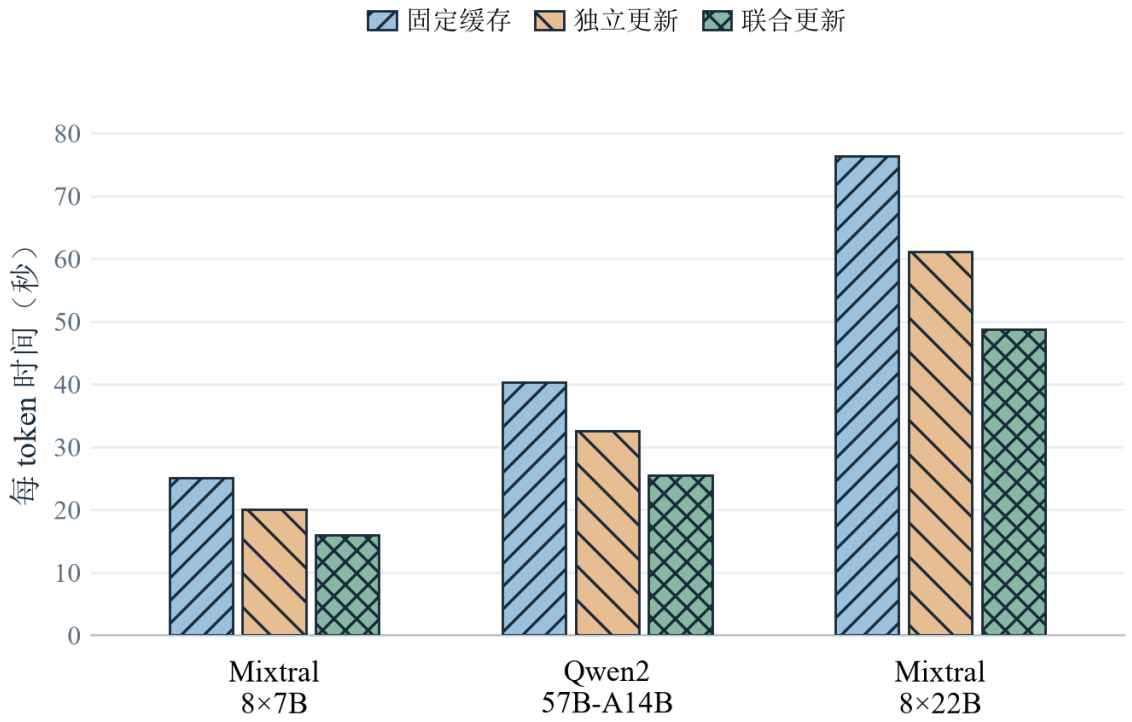
Insight III: 根据实际访问调整专家缓存

- 部署前的统计结果用于确定初始位置；推理时根据实际访问更新显存和 CPU 内存的缓存顺序。任务变化后，新任务频繁访问的专家逐步进入缓存，从而减少后续 HDD 读取。

设计示意：根据实际访问更新缓存

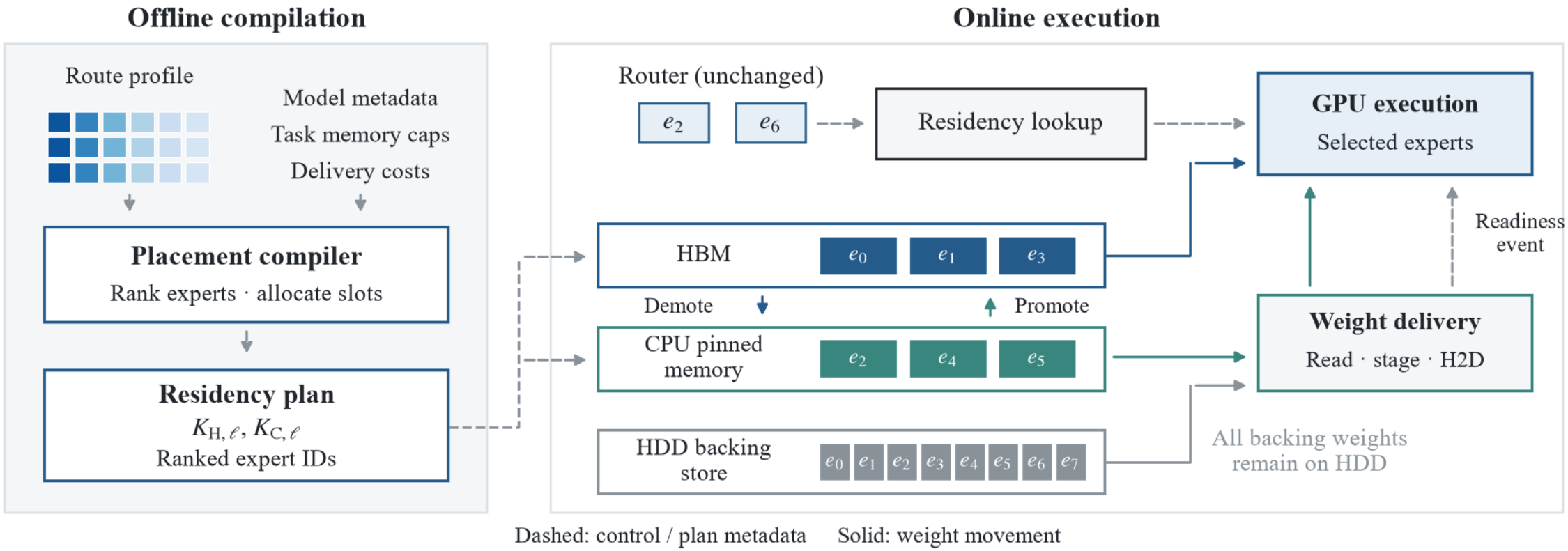


消融实验：比较任务变化后的表现



System Overview: MECC 系统总览

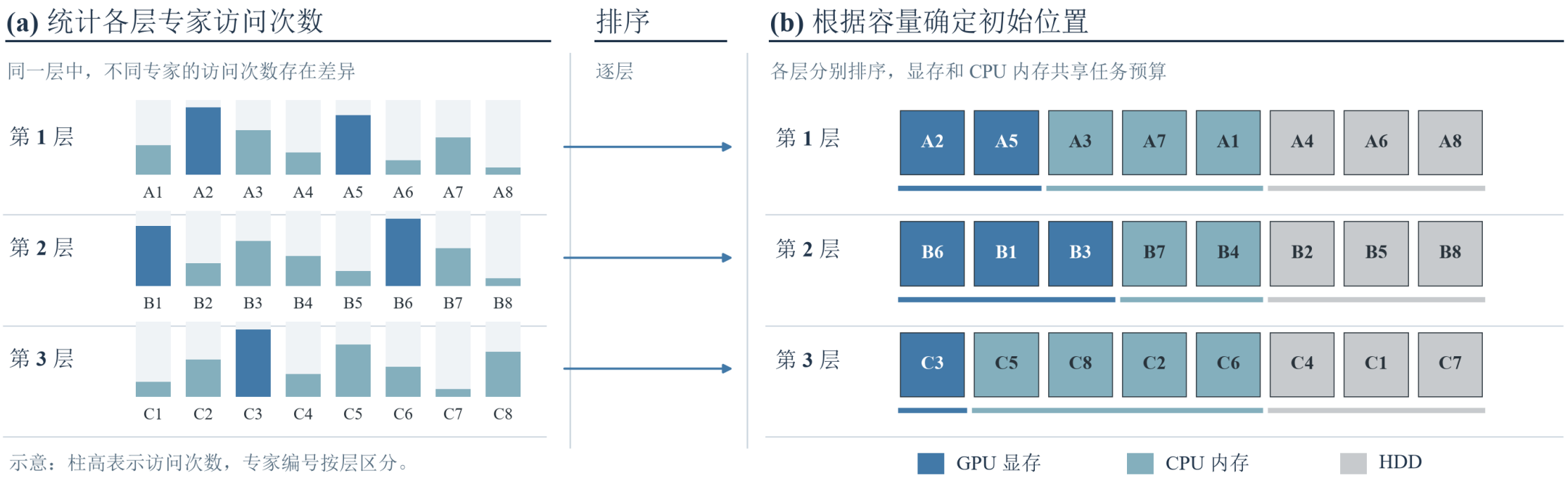
- MECC 在部署前确定专家的初始位置，在推理时根据实际路由访问并更新缓存。HDD 保存完整的专家权重；显存和 CPU 内存只缓存部分专家。所选专家的权重到达显存后执行计算。



部署前确定初始位置；推理时根据路由访问和更新缓存。

Method I: 确定专家的初始存储位置

- 部署前使用代表性输入统计每层专家的访问频率，并记录权重大小及任务的内存预算。
  编译器按层排序，在显存和 CPU 内存预算内确定初始位置；HDD 保存完整权重。



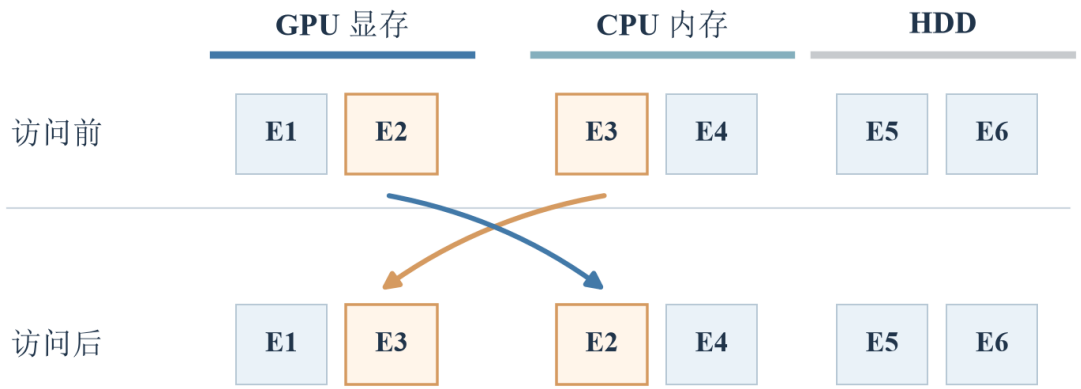
编译结果记录各层的缓存容量和专家编号；推理时继续根据实际请求更新缓存。

Method II: 更新缓存并执行专家计算

- 推理时先根据路由查找所选专家。CPU 内存命中时将专家传输到显存，并把被替换的专家保留在 CPU 内存；访问 HDD 时，等待权重到达显存后再执行计算。

(a) 一次 CPU 内存访问如何更新缓存

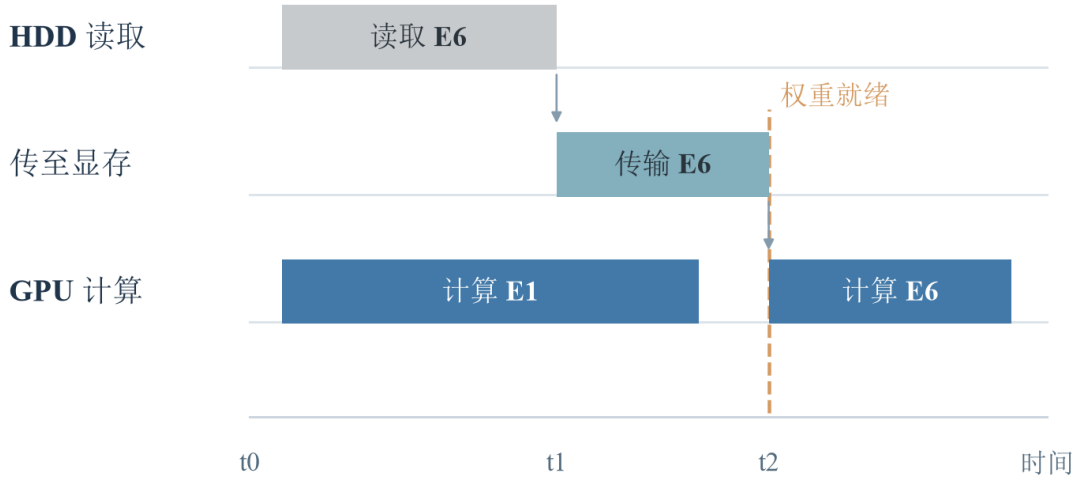
访问 E3 后，E3 进入显存；被替换的 E2 保留在 CPU 内存



E2 保留在 CPU 内存，后续访问无需再次读取 HDD。

(b) 所选专家何时可以计算

已在显存的 E1 先计算；E6 的权重到达后再执行

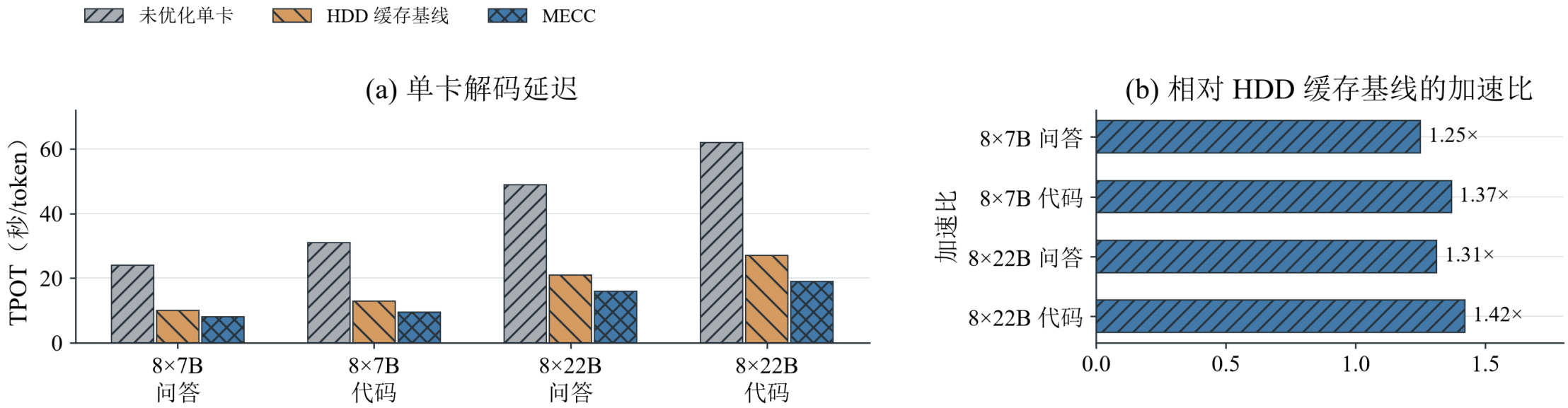


同一请求中，权重读取与已有专家的计算可以重叠。

两级缓存更新减少重复读取；已在显存的专家可先计算，其他专家在权重到达后计算。

Results: 单卡解码延迟

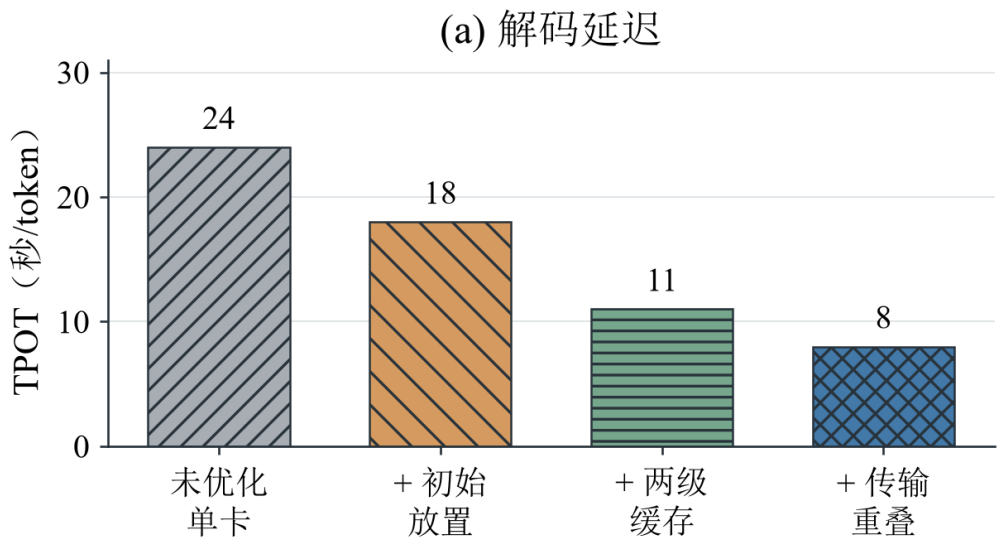
- 左图比较四组模型与任务中，未优化单卡方案、HDD 缓存方案和 MECC 的每 token 解码延迟。右图给出 MECC 相对 HDD 缓存方案的加速比。

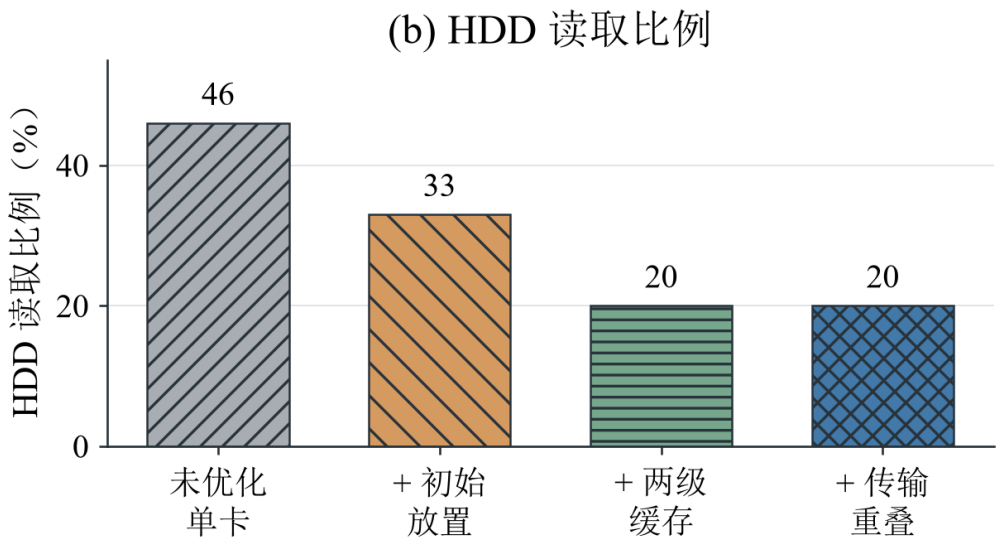


同一任务内存额度下，MECC 与 HDD 缓存基线相差 1.25–1.42×。

Ablation: 各项设计的效果

- 从基础方案开始，依次加入初始放置、两级缓存更新和计算与传输重叠。左图比较解码延迟，右图比较 HDD 读取比例，以区分各项设计对读取次数和等待时间的影响。

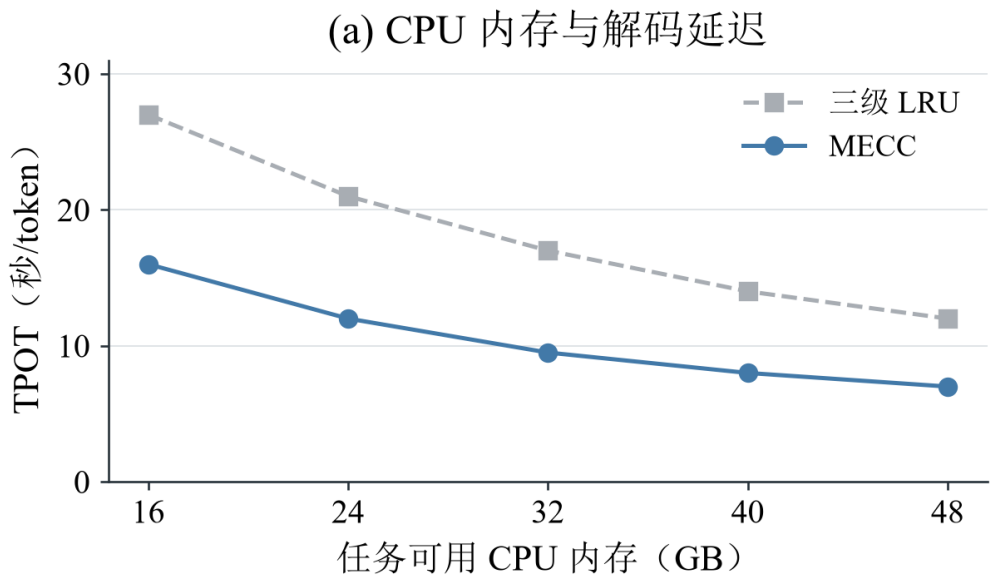


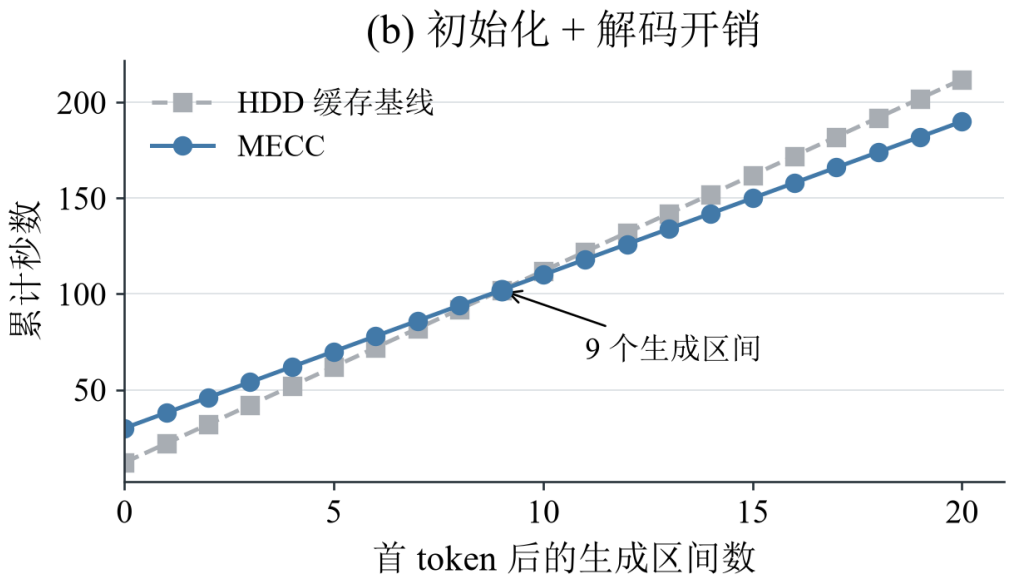


首组任务：24 → 18 → 11 → 8 秒/token；HDD 请求比例：46% → 33% → 20%。

Resource Sensitivity: CPU 内存与生成长度

- 任务可用的 CPU 内存越大，可缓存的专家越多。生成的 token 越多，初始加载开销对平均延迟的影响越小。两张图分别改变 CPU 内存预算和生成长度。

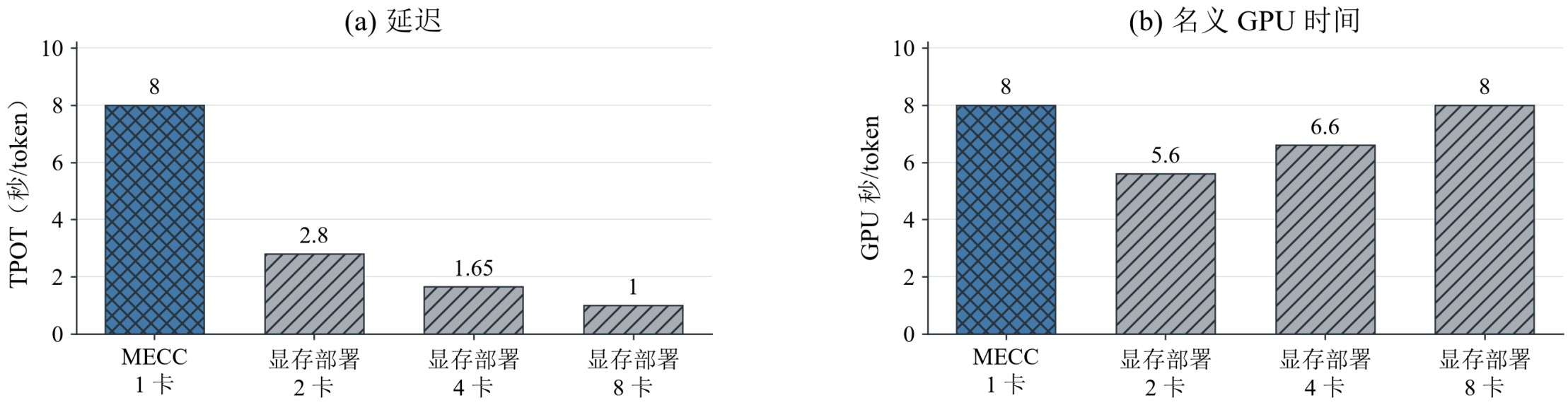




单卡推理性能受到 CPU 内存预算和生成长度共同影响。

Deployment Trade-off: 单卡与多卡部署

- 在总显存足以容纳模型的多卡配置中，专家分散驻留于 GPU，因此单 token 延迟更低；MECC 使用一张 GPU 运行完整模型。右图以 GPU 数量乘以延迟，比较两种部署的 GPU 资源开销。



单卡方案适用于 GPU 数量受限、可以接受较长推理延迟的任务。

Conclusion: 单卡运行完整 MoE 模型

- 当任务可用的 CPU 内存无法容纳全部专家时，部分权重需要保存在 HDD。MECC 依据访问频率确定初始位置，联合更新显存和 CPU 内存缓存，并根据实际请求调整，以减少 HDD 读取。

初始放置

优先缓存高频专家

减少首次读取

缓存更新

联合使用显存与 CPU 内存

减少重复读取

动态调整

根据实际访问更新缓存

适应任务变化

面向单卡、有限任务内存预算下的完整 MoE 推理。